

AADK 7: Task 2 — Input, Validation & UI

Feedback Plan

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 7

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Feature Scenario

Expense Splitter

The Expense Splitter allows users to enter a bill amount and the number of people sharing the expense. The app then calculates how much each person should pay. A toggle switch allows the user to round up the final amount per person.

This feature is useful because it simplifies splitting bills during group outings such as dining at restaurants, trips, or shared expenses.

2. Input & State Variables Listing

Input Field / UI Element	State Variable	Data Type	Validation Rule
Bill Amount (TextField)	billAmountInput	String / Double	Must be greater than 0
Number of People (TextField)	numPeopleInput	String / Int	Must be ≥ 1
Round Up Amount (Switch)	roundUpSwitch	Boolean	No validation required
Calculate Button	calculateEnabled	Boolean	Enabled only when inputs are valid

Example State Variables (Pseudo Code)

```
var billAmountInput by remember { mutableStateOf("") }  
var numPeopleInput by remember { mutableStateOf("") }  
var roundUpSwitch by remember { mutableStateOf(false) }  
var resultPerPerson by remember { mutableStateOf(0.0) }
```

3. UI Feedback & Validation States

Validation Rule	UI Feedback
Bill amount ≤ 0 or empty	Error text: "Bill amount must be greater than 0" displayed under TextField
Number of people < 1	Error text: "Enter at least 1 person"
Invalid number input	Field border turns red
Valid input	Error message disappears and state updates
Valid inputs for both fields	Calculate button becomes enabled

Behaviour

When Input is Valid

- State variables update.
- Calculation is performed.
- Result (per-person amount) is displayed.

When Input is Invalid

- Error message appears below the input field.
 - Calculate button remains disabled.
 - Result section remains hidden or unchanged.
-

4. Wireframe & Flow Sketch

Screen Layout (Wireframe)

Expense Splitter

Bill Amount

[TextField]

Error: Bill amount must be > 0

Number of People

[TextField]

Error: Enter at least 1 person

Round Up Amount

[Switch ON / OFF]

[Calculate Button]

Per Person Amount:

₹ 120.00

Composables Used

UI Element	Compose Component Modifiers	
Bill Amount Input	TextField	error state color
Number of People Input	TextField	error border
Round Up Option	Switch	toggle state
Calculate Button	Button	enabled / disabled
Result Display	Text	dynamic update

UI Flow

User enters bill amount



User enters number of people



Validation Check



If valid → Enable Calculate Button



User presses Calculate



State updated → Result displayed

5. Reflection

Validation and user feedback are important because they help users understand whether their inputs are correct and prevent incorrect calculations. In reactive UI frameworks like Jetpack Compose, state changes immediately affect the UI, so invalid data can easily propagate and cause incorrect results if not validated. Proper validation ensures that only correct data updates the application state. Without validation, users might see wrong calculations or the app may even crash due to invalid inputs such as dividing by zero. Therefore, validation improves both user experience and application reliability.
